



Universidad
Tecnológica
del Perú

PRIMERA UNIDAD DE APRENDIZAJE 2: Patrones creacionales

CURSO: DISEÑO DE PATRONES

SESIÓN N° 06:

Patrones Creacionales:

Introducción a patrones creacionales. Patrón Singleton. Patrón Prototype.

DOCENTES DE UTP
SISTEMAS



Lo que debemos tener en cuenta



5 minutos



¿QUÉ SON PATRONES CREACIONALES?

https://youtu.be/_KZkbL0MMbQ?feature=shared



¿Qué vimos en la sesión anterior?



1. ¿Cuál es la principal diferencia entre patrones arquitectónicos y patrones de diseño?

Select the correct answer

- ☐ a. Los patrones arquitectónicos son más abstractos
- ☐ b. Los patrones de diseño solo se aplican a software
- ☐ c. Los patrones de diseño son solo para arquitectos
- ☐ d. Los patrones arquitectónicos son específicos para interfaces

2. ¿Qué elemento es fundamental en un patrón de diseño?

Select the correct answer

- ☐ a. La creación de nuevas arquitecturas
- ☐ b. La implementación de lenguajes de programación
- ☐ c. La estructura de una base de datos
- ☐ d. La reutilización de soluciones probadas



3. ¿Qué patrón arquitectónico se utiliza comúnmente en aplicaciones web?

Select the correct answer

- ☐ a. Patrón de diseño Singleton
- ☐ b. Patrón arquitectónico MVC
- ☐ c. Patrón de diseño Observer
- ☐ d. Patrón arquitectónico de Microservicios

4. En el contexto de software, ¿qué es un patrón de diseño?

Select the correct answer

- ☐ a. Una solución abstracta a un problema recurrente
- ☐ b. Un tipo de base de datos relacional
- ☐ c. Un esquema para construir un edificio
- ☐ d. Un documento de requisitos funcionales



Unidad de aprendizaje 2:
Patrones creacionales.

Semana 5,6 y 7

Logro específico de aprendizaje:

Al finalizar la unidad el participante aplica los patrones creacionales para la solución de problemas.

Temario:

- Consolidación de temas (Repaso).
- Patrones Creacionales: Introducción a patrones creacionales. Patrón Singleton. Patrón Prototype.
- Patrones Creacionales: Patrón Factory. Patrón Abstract Factory. Patrón Abstract Builder.

Al final de la sesión el estudiante
aplica patrones creacionales como
Singleton y Prototype en un contexto
definido



La instanciación de clases o la generación de objetos es el foco de estos patrones de diseño. Los patrones de creación de clases y los patrones de creación de objetos son dos subconjuntos de estos patrones. Mientras que los patrones de creación de clases hacen un buen uso de la herencia en el proceso de instanciación, los patrones de creación de objetos utilizan la delegación.

Factory Method, Abstract Factory, Builder, Singleton, Object Pool y Prototype son patrones de diseño de creación.



¡Que no se te escape nada de tu clase!:

SESIÓN N° 06:

Patrones Creacionales:
Introducción a patrones creacionales.
Patrón Singleton. Patrón Prototype.

Los patrones creacionales son un tipo de patrón de diseño que se enfocan en la creación de objetos.

Estos patrones abstractizan el proceso de instanciación de objetos y ayudan a hacer el sistema independiente de cómo se crean, componen y representan los objetos.

Los patrones creacionales permiten al sistema delegar la responsabilidad de creación a otros objetos, facilitando la flexibilidad y la reutilización del código.

Los patrones creacionales más comunes son:

1. Singleton
 2. Prototype
 3. Factory Method
 4. Abstract Factory
 5. Builder
-

Introducción a patrones creacionales – caso de uso

Supongamos que un programador desea crear una clase `DBConnection` simple para conectarse a una base de datos y necesita usar la base de datos desde el código en varios lugares.

El desarrollador normalmente creará una instancia de la clase `DBConnection` y la usará para realizar operaciones de base de datos donde sea necesario.

Como cada ejemplo de la clase `DBConnection` tiene una conexión diferente a la base de datos, se crean numerosas conexiones a la base de datos.

Para solucionarlo, hacemos que la clase `DBConnection` sea una clase singleton, lo que significa que solo se genera una instancia de `DBConnection` y solo se realiza una conexión.

Podemos controlar el equilibrio de carga, las conexiones redundantes, etc., ya que podemos administrar `DBConnection` desde una sola instancia.

DatabaseConnection

- instance: DatabaseConnection
- connection: Connection
- url: String
- username: String
- password: String

- + getInstance(): DatabaseConnection
- DatabaseConnection()
- + getConnection(): Connection

El patrón Singleton es un patrón de diseño de creación y uno de los patrones de diseño más fundamentales que podemos utilizar.

Es un método para proporcionar un único objeto de un tipo particular. Simplemente se necesita una clase para definir métodos e identificar objetos.

Según el patrón Singleton, “crea una clase que tenga una única instancia y proporcione un punto de acceso global”.

En otras palabras, una clase debe garantizar que se cree una única instancia y que todas las demás clases puedan acceder a un único objeto.

El patrón de diseño Singleton se presenta en dos variedades.

- Instanciación temprana: la construcción de una instancia en el momento de la carga.
- Instanciación perezosa: creación de instancias solo cuando es necesario.

Introducción a patrones creacionales – Singleton

El patrón Singleton asegura que una clase tenga solo una instancia y proporciona un punto de acceso global a dicha instancia. Es útil cuando se necesita exactamente un objeto para coordinar acciones en todo el sistema.

Implementación en Java

```
public class Singleton {  
    // Instancia única de la clase  
    private static Singleton instance;  
    // Constructor privado para evitar instanciación directa  
    private Singleton() {}  
    // Método público para obtener la instancia única  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Singleton

- uniqueInstance: Singleton

+ getInstance(): Singleton

Introducción a patrones creacionales – Singleton – Ventajas y desventajas

Ventajas

Inicializaciones: Un objeto generado por el Patrón Singleton solo se inicializa la primera vez que se lo solicita.

Obtención de acceso al objeto: Obtuvimos acceso global a la instancia del objeto.

Cantidad de ocurrencias: Las clases que usan el Método Singleton solo pueden tener una instancia.

Desventajas

Un entorno con múltiples subprocesos: resulta complicado utilizar el patrón Singleton en un entorno de múltiples subprocesos porque debemos asegurarnos de que el multiproceso no construya el objeto Singleton varias veces.

Principio de responsabilidad exclusiva: debido a que la técnica Singleton resuelve dos problemas a la vez, viola la idea de responsabilidad única.

Procedimiento para pruebas unitarias: debido a que agregan un estado global al programa, las pruebas unitarias se vuelven muy difíciles.

Introducción a patrones creacionales – Ejemplo Java

```
public class Singleton {  
    // Crear una instancia estática de la clase Singleton  
    private static Singleton uniqueInstance;  
    // Constructor privado para evitar la instanciación externa  
    private Singleton() {  
        // Código de inicialización  
    }  
    // Método público para obtener la única instancia de la clase  
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            uniqueInstance = new Singleton();  
        }  
        return uniqueInstance;  
    }  
    // Método de ejemplo  
    public void showMessage() {  
        System.out.println("Hello World from Singleton!");  
    }  
}
```

```
// Clase de demostración para probar el patrón  
Singleton  
public class SingletonDemo {  
    public static void main(String[] args) {  
        // Obtener la única instancia de Singleton  
        Singleton singleton = Singleton.getInstance();  
        // Llamar al método de la instancia Singleton  
        singleton.showMessage();  
    }  
}
```

Explicación:

1. `**`Singleton` class**`:

- `**Instancia única**`: La variable ``uniqueInstance`` se declara como ``private static`` para asegurar que solo haya una instancia de la clase.
- `**Constructor privado**`: El constructor es privado para prevenir la creación de nuevas instancias desde fuera de la clase.
- `**Método `getInstance`**`: Este método proporciona un punto de acceso global a la única instancia de la clase. Si la instancia no existe, se crea; si ya existe, se devuelve la instancia existente.
- `**Método `showMessage`**`: Un método de ejemplo que imprime un mensaje.

2. `**`SingletonDemo` class**`:

- `**`main` method**`: Este método obtiene la única instancia de ``Singleton`` usando el método ``getInstance`` y llama al método ``showMessage`` para demostrar su uso.

Introducción a patrones creacionales – Prototype

El patrón Prototype se usa para crear nuevos objetos copiando una instancia existente, conocida como prototipo. Este patrón es útil cuando la creación directa de un objeto es costosa o compleja.

Implementación en Java

```
public class Prototype implements Cloneable {  
    private String attribute;  
    public Prototype(String attribute) {  
        this.attribute = attribute;  
    }  
    // Método para clonar el objeto  
    @Override  
    public Prototype clone() throws CloneNotSupportedException {  
        return (Prototype) super.clone();  
    }  
    public String getAttribute() {  
        return attribute;  
    }  
    public void setAttribute(String attribute) {  
        this.attribute = attribute;  
    }  
}
```

Introducción a patrones creacionales – Prototype

```
// Uso del patrón Prototype
public class Main {
    public static void main(String[] args) {
        try {
            Prototype prototype1 = new Prototype("Attribute 1");
            Prototype prototype2 = prototype1.clone();
            prototype2.setAttribute("Attribute 2");
            System.out.println(prototype1.getAttribute()); // Output: Attribute 1
            System.out.println(prototype2.getAttribute()); // Output: Attribute 2
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}
```

Introducción a patrones creacionales – Prototype – ejemplo en Java

```
public abstract class Shape implements Cloneable {  
    private String id;  
    protected String type;  
    abstract void draw();  
    public String getType() {  
        return type;  
    }  
    public String getId() {  
        return id;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
    @Override  
    public Object clone() {  
        Object clone = null;  
        try {  
            clone = super.clone();  
        } catch (CloneNotSupportedException e) {  
            e.printStackTrace();  
        }  
        return clone;  
    }  
}
```

Introducción a patrones creacionales – Prototype – ejemplo en Java

```
}  
}  
public class Rectangle extends Shape {  
    public Rectangle() {  
        type = "Rectangle";  
    }  
    @Override  
    void draw() {  
        System.out.println("Inside Rectangle::draw() method.");  
    }  
}  
public class Circle extends Shape {  
    public Circle() {  
        type = "Circle";  
    }  
    @Override  
    void draw() {  
        System.out.println("Inside Circle::draw() method.");  
    }  
}
```

```
public class PrototypePatternDemo {  
    public static void main(String[] args) {  
        // Crear instancias de prototipos  
        Rectangle rectangle = new Rectangle();  
        rectangle.setId("1");  
        Circle circle = new Circle();  
        circle.setId("2");  
        // Clonar los prototipos  
        Shape clonedRectangle = (Shape) rectangle.clone();  
        Shape clonedCircle = (Shape) circle.clone();  
        System.out.println("Shape : " + clonedRectangle.getType());  
        clonedRectangle.draw();  
        System.out.println("Shape : " + clonedCircle.getType());  
        clonedCircle.draw();  
    }  
}
```

Explicación:

1. **`Shape` class`**:

- **`Atributos`**: `id`` y `type`` son atributos comunes a todas las formas.
- **`Métodos abstractos`**: `draw()` es un método abstracto que será implementado por las clases concretas.
- **`Método clone``**: Sobrescribe el método `clone`` de la clase `Object`` para permitir la clonación de objetos.

Maneja `CloneNotSupportedException`` que puede ser lanzada por `super.clone()`.

2. **`Rectangle` class`** y **`Circle` class`**:

- Extienden `Shape`` y proporcionan implementaciones concretas del método `draw()`.

3. **`PrototypePatternDemo` class`**:

- **`main` method`**: Crea instancias de `Rectangle`` y `Circle``, luego las clona usando el método `clone`` y llama a `draw`` para demostrar su uso.

¿Cuál es la diferencia entre Singleton y Prototype?



Caso práctico



Realizar ejercicio práctico es una excelente manera de aprender y entender cómo elaborar un patrón de diseño creacional. Aquí tienes un ejercicio práctico detallado que puedes seguir:

Ejercicio Práctico:

Caso de Uso: Aplicación de gestión de pedidos

En una aplicación de gestión de pedidos, se implementa un gestor de conexiones a la base de datos utilizando el patrón Singleton para garantizar que solo exista una única instancia de la conexión a la base de datos en toda la aplicación. Esto permite que diferentes componentes de la aplicación accedan a la base de datos de manera eficiente y centralizada, evitando la creación de múltiples conexiones y mejorando el rendimiento general de la aplicación.

Objetivos:

1. Elaborar el diagrama de clases UML utilizando el patrón seleccionado.



¿Quién quisiera participar? (2 voluntarios)





¿Qué se les hizo más fácil?
¿Qué se les hizo más retador?





¿Qué hemos aprendido el día de hoy?



Tomar apuntes de manera eficaz ayuda a consolidar el aprendizaje y prepararse para los exámenes.



La creación de instancias de clases es fundamental para varios patrones de diseño.

Este patrón se divide en dos tipos: patrones de creación de clases y patrones de creación de objetos.

Mientras que los patrones de creación de clases emplean la herencia en el proceso de creación de instancias, los patrones de creación de objetos utilizan de manera eficaz la delegación.



- Elabora la actividad práctica de acuerdo a la guía de laboratorio de sesión
- Sube la actividad práctica en la plataforma virtual de aprendizaje
- Guarda la actividad con la siguiente etiqueta:
DPA_Actividad06_NombreApellido

Nota: No olvides también revisar tu plataforma UTP+Class



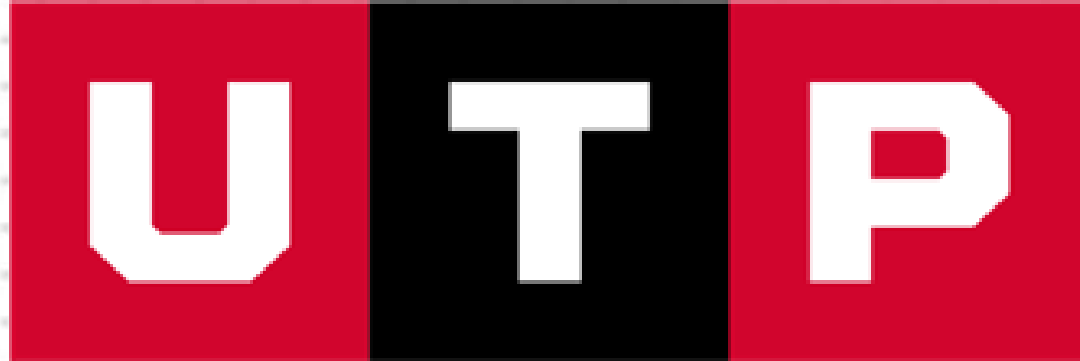
- Patrones creacionales(2022). Ecosistema de recursos educativos digitales. https://youtu.be/_KZkbL0MMbQ?feature=shared
- bin Uzayr, S. (2023). Software Design Patterns: The Ultimate Guide. CRC Press.

Nota: No olvides también revisar tu plataforma UTP+Class



**MUCHAS GRACIAS QUE
DIOS LOS BENDIGA!!!**





**Universidad
Tecnológica
del Perú**

